

The publication process

Kjell Post

2026

Outline

- Scientific articles
- How papers get reviewed
- A skeleton paper
- Presenting your material
- A collection of advice
- Giving presentations
- Tools

Question authority. Listen to what I have to say, and see whether it makes sense to you.

Scientific articles

- **Journal papers** — new results, or surveys.
- **Conference papers** — full papers, short papers, posters.
In computer science these are peer-reviewed and count as real publications; in most other fields they do not.
- **Technical reports** — published by a university or company, not peer-reviewed.
- **Preprints** — arXiv and friends. Not reviewed, citable, permanent, and increasingly how people actually read your work.
- **Magazines** — *Communications of the ACM*, *IEEE Spectrum*, *ACM Queue*. Written for a broad audience.
- **Theses** — your licentiate or doctoral thesis, often a cover essay plus the papers themselves.

Author's goal

- To describe interesting new knowledge in an understandable way.
- It must describe previously unpublished work.

You can learn a great deal from studying other people's work, but . . .

Never, ever, claim somebody else's work as your own.

This includes re-publishing your *own* earlier work as if it were new. Publishers run submissions through similarity checkers, and self-plagiarism is caught by the same tools as any other kind.

Ask yourself

- What is the problem I have solved?
- Which method did I use?
- Is my work new and relevant?
- What are my contributions?
- Who is going to read this?

The reader should be

- *intrigued* within the first 5 minutes of reading;
- *excited* within 15 minutes;
- *satisfied* after 45 minutes.

Where to submit

- ACM and IEEE sponsor most CS and CE journals and conferences.
- Find the deadlines: the venue's own call for papers, WikiCFP, DBLP, and whatever mailing list your field uses.
- Decide where to submit *before* writing the paper — you may need to slant it for a particular audience, and the page limit shapes the paper.
- Important dates for a conference:
 - abstract deadline, then full submission;
 - author response (“rebuttal”) period;
 - notification: *reject* or *accept*;
 - camera-ready copy;
 - the conference itself.
- Journals have a much longer pipeline, often with major revisions.
- Read the “Instructions to Authors”. All of them.

Choosing a venue: a warning

- **Predatory venues exist.** They will accept anything, charge you a fee, and give you a line on your CV that does you harm.
- Warning signs: flattering spam invitation, a promise to review in 72 hours, a conference with no identifiable programme committee, a journal whose scope is “multidisciplinary” and unbounded.
- Check whether the people on the programme committee are people you have heard of, and whether the venue appears in DBLP.
- **Open access** is a separate question from quality. Many excellent venues charge an article processing charge; ask whether your funder or university pays it *before* you submit.
- Ask your supervisor. This is exactly what they are for.

How papers get reviewed

- **Single-blind** — reviewers know who you are.
- **Double-blind** — they do not, and this is now the norm at many CS conferences. It changes how you write:
 - no names or affiliations on the submitted version;
 - cite your own earlier work in the third person (*Post et al. showed . . .*), not as *our earlier work*;
 - do not link to a repository that carries your name.
- **Open review** — reviews, and sometimes reviewer names, are published alongside the paper.
- Reviewers are volunteers, working to a deadline, reading your paper late at night. Make their job easy.
- You will be asked to review too. Do it, and do it properly — write the review you wish you had received.

Dealing with page limits

- Conferences almost always have a page or word limit.
- Don't cheat with tiny fonts, narrow margins or squeezed line spacing. The chairs check, and papers get desk-rejected for it.
- Eliminate redundant material first. Trim the technical section last.
- Move detail to an appendix or a technical report, and cite it — many venues now allow a clearly marked appendix that reviewers may read at their discretion.
- Count words properly: `texcount paper.tex` understands $\text{\LaTeX}$ markup; `wc -w` counts your macros as words.

Rejections

Papers get rejected for many reasons.

- Many good conferences accept 15–25% of submissions.
- Your paper may simply not “lie within the scope”.

But you get reviews out of it, and those are valuable.

- Think twice before complaining. Sleep on it. Then think again.
- Reviewers are anonymous. Don't try to guess who they were.
- Use the criticism constructively: if one reviewer misunderstood something, other readers will too. That is a writing problem, and it is yours.
- If there is an author response period, answer the factual errors and the most damaging misunderstanding. Be brief and be polite; you are talking to someone who will read the next version.
- Then consider sending it somewhere else — improved.

Things you now have to declare

Every venue has a policy page. Read it before you submit.

- **Conflicts of interest** — co-authors, recent collaborators, your own institution.
- **Authorship** — who qualifies, and in what order. Agree this early, in writing, before anybody is annoyed. Some venues ask for a per-author contribution statement.
- **Data and code availability** — many venues have artifact evaluation, and award badges for code that others can actually run.
- **Ethics approval** for anything involving human subjects, and a broader-impact statement in some fields.
- **Generative AI** — publishers now require you to disclose how you used it, and it cannot be an author. The rules differ per venue and they change; check the current policy, not this slide.
- Get an **ORCID**. It is free and it stops you being confused with the other person who shares your name.

A skeleton paper

Title
Author(s)
[Contents, list of tables, figures]
Abstract
[Index terms / keywords]
Introduction
 Background information
 Problem definition
 Summary of contributions
 Related work
Method
Results
Summary and conclusions

[Data and code availability]
Acknowledgements
References
Appendix
[Index]

But don't write the paper in this order.

Most people write the method and results first, the introduction late, and the abstract last.

The title

- Must be informative, clear and meaningful.
- Don't be clever or cryptic.
- Get the reader's attention immediately.
Bad: The effects of stress
Good: Is stress killing you?
Or: Stress: is it killing you?
- Don't use symbols or unexplained acronyms in titles.
- Imagine someone searching for your paper. The words that would bring them to it should be in the title.
- It is also what people see on a slide, in a reference list, and in a tweet. Make it survive all three.

The abstract (read by 87%)

- Summarises the problem, the result and its uses.
- Between 100 and 250 words.
- Avoid references and acronyms.
- Try to *sell* the paper.

"A pointer-analysis algorithm can be either flow-sensitive or flow-insensitive. While flow-sensitive analysis usually provides more precise information, it is also usually considerably more costly in terms of time and space. The main contribution of this paper is the presentation of another option in the form of an algorithm that can be 'tuned' to provide a range of results that fall between the results of flow-insensitive and flow-sensitive analysis. The algorithm combines a flow-insensitive pointer analysis with static single assignment (SSA) form and uses an iterative process to obtain progressively better results."

Keywords, and the introduction (43%)

Index terms / keywords

- Used to *classify* your paper, and to find it.
- ACM has a concept classification scheme; many venues ask for it.

Introduction

- Explains the background and the significance of the paper.
- The opening paragraph should be your best paragraph. The opening sentence should be your best sentence.

Bad: *An important method for internal sorting is quicksort.*

Good: *Quicksort is an important method for internal sorting, because ...*

- End it with a summary of how the paper is organised.

Background and problem definition

Background information

- A short introduction to the area — “A brief review of LR parsing”.
- Cite references for the details.

Problem definition

- A concise statement of the problem you are solving.
- Why is it useful to solve it?

If your problem is “develop an X algorithm capable of handling very large Y problems in reasonable time”, then:

- explain what “large” and “reasonable” mean here;
- show that existing algorithms fail;
- justify why a solution would be useful.

Summary of contributions

- What was investigated or built, and what you found out.
- Don't make forward references into the technical section.

You are writing a scientific paper, so:

- opinions must be backed by arguments or references — otherwise remove them;
- facts must be verifiable by the reader;
- be humble, and don't use superlatives.

Bad: *This is a great algorithm.*

Good: *We believe this algorithm is useful for the following reasons.*

- a numbered list of contributions at the end of the introduction is a cliché, and it works. Reviewers look for it.

Related work

- Identify *all* related work, with references — even work that is old, or that solves a different problem.
- References must be locatable, so avoid “personal communication”. Give a DOI or a stable URL where you can.
- Compare your work with previous work. You must convince the reader that yours is original. Examples and measurements do this well.
- Clearly separate *your* results from *their* results.
- Don't be condescending about other people's work. The author is quite likely reviewing your paper.
- Missing the obvious related paper is one of the commonest reasons for rejection. Search properly, and search again just before you submit.

Method (12%) and results

Method — how you did it.

- Tests, procedures, experiments, equipment, data structures, algorithms.
- Proofs for algorithms: termination, and correctness (soundness, completeness).
- Don't describe dead ends.
- Write it so that somebody else could repeat it. That is the whole point.

Results

- Benchmarks; time and space complexity.
- Say what the baseline is, how many runs, and how much the numbers vary. A single number with no spread is not a result.

Summary and conclusions (55%)

- Interpretation of your work: the pros and the cons.
- Limitations of the method — state them yourself.
- Suggestions for future work.
- Rejected alternatives, and what you learned.
- Pointers to your software and data.
- For the conclusion, don't just re-word the abstract. You may refer to figures and equations here.

Acknowledgements

- Mention anyone who helped, including reviewers of earlier versions.
- Name your funding, with the grant number. Your funder requires it.

References

- Make sure every reference is actually cited, and every citation actually resolves.
- Common citation styles:
Turing stated the following theorem [12] ...
... as shown by Ullman [Ull87].
- What goes into an entry depends on the type of publication:
 - **Book:** author, title, publisher, year, ISBN.
 - **Journal:** author, title, journal, volume, number, pages, year, DOI.
 - **Report:** author, title, institution, number, year.
 - **Preprint:** author, title, archive and identifier, year.
- Keep the entries in a `.bib` file and let `biblatex` format them. Never hand-format a reference list.

On the next slide I cite an article by Debray [DW88], a technical report and a conference paper by Warren [War83; War86], and a book by Lloyd [Llo84].

- [DW88] Saumya K. Debray and David S. Warren. “Automatic Mode Inference for Logic Programs”. In: *The Journal of Logic Programming* 5 (1988), pp. 207–229.
- [Llo84] J. W. Lloyd. *Foundations of Logic Programming*. Springer-Verlag, 1984.
- [War83] D. H. D. Warren. *An Abstract Prolog Instruction Set*. Tech. rep. Tech. Note 309. SRI International, Menlo Park, CA, 1983.
- [War86] D. H. D. Warren. “Optimizing Tail Recursion in Prolog”. In: *Logic Programming and its Applications*. Ablex Publishing, N.J., 1986, pp. 77–90.

Paragraph structure

A paragraph should have

- a clear, well-defined purpose;
- a topic sentence that gives the main subject of the paragraph;
- a mix of long and short sentences.

“Latin abbreviations like i.e. and e.g. are best avoided. If you must use them, punctuate them properly (i.e., like this). I prefer to use the simple English phrases ‘in other words’ and ‘for example’ instead.”

- Avoid contentless beginnings: *There are, There is, It is, This, These.*
- Avoid rambling paragraphs, and avoid artificial breaks.

Figures

- For pseudo-code, data structures, architecture, progress charts.
- A figure floats; refer to it by number (Figure 1), never as “the figure below”.
- The caption explains the figure and can be read on its own.

Bad: *Figure 3. Step 2 of the algorithm.*

Good: *Figure 3. Swapping out-of-order data elements pointed to by i and j .*

append([], L_2 , L_2).

append($[X|L_1]$, L_2 , $[X|L_3]$) $\leftarrow$ *append*(L_1 , L_2 , L_3).

Figure: The call *append*(L_1 , L_2 , L_3) succeeds when L_3 is the result of appending L_1 to L_2 .

Displays

- For mathematical formulas and very small code fragments.
- A display is set in a fixed place, unlike a float.

Prove by induction that

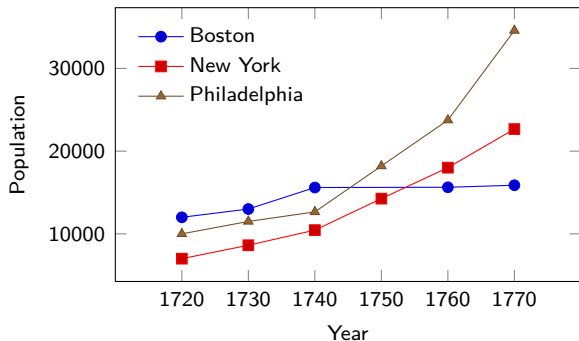
$$\sum_{k=1}^n k = \frac{n(n+1)}{2}$$

for all $n \geq 1$.

Use `\[...\]` for a display, and `equation` when you need to number it. Not `$$...$$` — see the $\text{\LaTeX}$ lecture.

Presenting graphs

- Use the caption to point out what the reader should notice.
- Label both axes, and give the units.
- No grid — use tick marks.
- Make sure the ranges are appropriate.
- Don't clutter the graph with decoration.
- Check it in greyscale, and don't rely on colour alone to distinguish the series.



Presenting algorithms

- Use pseudo-code, without full detail.
- Clearly show the input and the output.

Input: a graph (V, E) and a source vertex s .

Output: the distance $d[u]$ from s , for every vertex u .

```
BFS( $s$ )  
  for each  $u \in V - \{s\}$   
    do  $d[u] \leftarrow \infty$   
 $d[s] \leftarrow 0$   
 $Q \leftarrow \{s\}$   
while  $Q \neq \emptyset$   
  do remove  $u$  from  $Q$   
    for each  $v \in Adj[u]$   
      do if  $d[v] = \infty$   
        then  $d[v] \leftarrow d[u] + 1$   
           put  $v$  onto  $Q$ 
```

Presenting examples

- What is the point of the example? If you cannot say, cut it.
- Examples that do *not* yield the desired result can be very useful.
- One running example beats a couple of unrelated ones.
- Use *realistic* examples. `foo` and `bar` teach nobody anything.
- Introduce the example before the formalism, not after it.

Presenting proofs

- State the theorem or lemma clearly first.
- Give the simplest proof possible. Don't use proof by contradiction if a direct proof exists.
- Explain the strategy of the proof before giving it.
- Clearly mark where the proof ends. $\square$

Some useful proof techniques:

Definitions and axioms	$A \wedge (A \rightarrow B) \rightarrow B$
Contrapositive	$(A \rightarrow B) \equiv (\neg B \rightarrow \neg A)$
Contradiction	$(\neg A \rightarrow \text{false}) \rightarrow A$
Case analysis	$((A \rightarrow B) \wedge (\neg A \rightarrow B)) \rightarrow B$
Construction	useful for showing $(\exists X) p(X)$
Induction	useful for showing $(\forall X) p(X)$

A proof, in prose

Proposition. If $f'(x) > 0$ for all x in an interval, then f is increasing on the interval.

Proof. Let a and b be two points in the interval with $a < b$. We must prove that $f(a) < f(b)$. By the Mean Value Theorem, there is some x in (a, b) with

$$f'(x) = \frac{f(b) - f(a)}{b - a}$$

But by hypothesis $f'(x) > 0$ for all x in (a, b) , so

$$\frac{f(b) - f(a)}{b - a} > 0$$

Since $b - a > 0$ it follows that $f(b) > f(a)$. $\square$

The same proof, structured

Proof. Let a and b be two points in the interval with $a < b$. We must prove that $f(a) < f(b)$.

<i>Statement</i>	<i>Reason</i>
1. There exists x in (a, b) with $f'(x) = \frac{f(b)-f(a)}{b-a}$	the Mean Value Theorem
2. $f'(x) > 0$	by 1 and the hypothesis
3. $\frac{f(b)-f(a)}{b-a} > 0$	by 1 and 2
4. $b - a > 0$	by the choice of a and b
5. $f(b) > f(a)$	by 3 and 4

Structured proofs are longer, and much harder to get away with being wrong in.

A collection of advice

1. Start early. Writing takes time.
2. Don't think about formatting while writing.
3. Don't start a sentence with a symbol.

Bad: *2 is the only even prime.*

Good: *The only even prime is 2.*

4. Always strive for complete sentences.

Bad: *We now have the following*

Theorem. *$H(x)$ is continuous.*

Good: *We can now prove the following result.*

Theorem. *The function $H(x)$ is continuous.*

A collection of advice

5. Place words between adjacent formulas.

Bad: *Add p k times to c .*

6. Use the symbols $\exists, \forall, \in$ only in logic.

Bad: $C \subset P \Rightarrow C \subset A_n$

Good: *C is a subset of A_n whenever C is a subset of P .*

In general, avoid unnecessary jargon.

7. Avoid subscripts if possible.

Bad: p_i is an element of P . **Good:** p is an element of P .

8. State the types of your variables: *the set S , the function F .*

9. Don't use the same notation for two different things.

A collection of advice

10. Stick to consistent usage. If $i = 1 \dots m$ and $j = 1 \dots n$, then **Bad**: $i < j$ and $p_j < p_i$.
11. Prefer the active voice. “We” is standard for joint work; for a single-author paper “I” is now perfectly acceptable, and better than contorting the sentence.
Bad: *The following results can now be proved.*
Good: *We can now prove the following result.*
12. Vary sentence structure and word choice.
13. Motivate the reader for what follows.
 - What does the reader know so far?
 - What does the reader expect next, and why?
14. Always avoid ambiguity.
“Optimal font caching”, “the node x is left” ... but “random number generator” is fine.

A collection of advice

15. Spell out small numbers used as adjectives.
The method requires two passes. Method 2 is illustrated in Figure 1.
16. Don't mix hyphens, dashes and minus signs: on-line, Oct 12–26, $x - y$.
The em dash is longer—like this—but is not used in Swedish.
17. When using “it”, make sure it is clear what *it* refers to.
18. Avoid frequent repetition of “this”, “they”, “just” and “then”.
Bad: *This is done by ...* **Good:** *This step is done by ...*
19. Number only the equations you refer to.
20. Never *display* a false equation or statement.

A collection of advice

21. Don't overuse typographic tools.
 - One proportional font for text, one fixed-width font for code.
 - Body text between 10 and 12 pt.
 - Colour is fine now that papers are read on screen — but it must still work in greyscale and for colour-blind readers, so never let colour be the only cue.
22. Use *italics* for emphasis, and when a term is first defined.
23. Let the document rest for a couple of days before polishing it.
24. Have someone read your draft before you submit it.
25. Read books by great authors. It will sharpen your style.
26. Keep the paper in version control, and tag what you submitted — you will want to diff it against the camera-ready version later. `latexdiff` makes that readable.

Giving presentations

- Tell them what you are going to tell them.
- Tell them.
- Then tell them what you told them.

—Ian Parberry

What is the purpose of your talk?

- Not to go through the paper.
- Rather, to make people read the paper later.

Ask yourself

- Who will be listening?
- What is the one thing you want them to remember?

Don't ...

- Don't impress the audience with hairy theory — at most 1–2 technical slides, and plenty of motivating examples.
- Don't present more material than you have to. Spend the detail on *one* thing.
- Don't hide the problems with your method. They will find out.
- Don't start with an outline; use “signposts” instead.
- Don't read from your slides. Talk about what is on them.
- Don't fiddle with the slides while you talk.

Timing

- Rehearse out loud, with a clock. Never *ever* run over.
- Know which slides you will drop if you are running late.

Slides, and the room

- 16:9, at least 20 pt body text, one idea per slide, numbered slides.
- Bring the talk as a *PDF*, on the laptop, a USB stick and somewhere online.
- Test the adapter on that projector, in the break before your session.
- Avoid live demos and anything that needs the conference wi-fi.
- For an online talk: headset microphone, camera at eye level, share one window, and assume it is being recorded.

The full version of this section is in `present-2026.tex`.

The question and answer session

Three kinds of question:

1. **The self-admirer**

“Have you read Dr. Smith’s paper?” — where Dr. Smith is the questioner. Thank them, say you will look it up, move on.

2. **The attacker**

“Your work is boring / irrelevant / contrived.”

Stay polite, answer in one sentence, offer to continue afterwards. The room is on your side as long as you are calm.

3. **Genuine interest**

This is the one you came for. And say “I don’t know” when you don’t — it costs nothing and buys a great deal.

Repeat the question before answering: half the room did not hear it.

Spelling and grammar

The 1998 version of this slide recommended `ispell` and a page at UCLA. Both are gone. Today:

- `hunspell -t -d en_GB paper.tex` — the `-t` flag makes it skip $\text{\LaTeX}$ markup. `aspell -t -c` does the same job. Swedish dictionaries are packaged as `sv_SE`.
- `chktex paper.tex` and `lacheck paper.tex` — both ship with TeX Live and catch $\text{\LaTeX}$ mistakes a spell checker cannot: wrong dashes, missing `~` before `\ref`, space before a footnote.
- Grammar and style: LanguageTool, and the L^AT_EX extension that puts it inside your editor with $\text{\LaTeX}$ awareness.
- `texcount paper.tex` — word counts that ignore markup.
- `latexdiff old.tex new.tex > diff.tex` — a marked-up PDF of what changed. Reviewers and supervisors love it.

None of these replaces reading the paper aloud.

Making graphs

- **pgfplots** — the plot on the earlier slide is drawn by $\text{\LaTeX}$ itself, straight from a .csv:

```
\begin{axis}[xlabel={Year}, ylabel={Population}]  
  \addplot table [x=year, y=boston, col sep=comma] {pop.csv};  
\end{axis}
```

It uses the document's fonts and maths, and the data file stays the source of truth.

- **gnuplot** still exists and is still good. Use a modern terminal — `pdfcairo`, or `cairolatex` so that the labels are typeset by $\text{\LaTeX}$. Not set `term postscript`.
- **matplotlib** or **R**, if the figure falls out of your analysis anyway. Export PDF, not PNG.
- Whatever you use, script it. A figure you cannot regenerate is a figure you cannot fix.

On your desk(top)

- **Dictionary and thesaurus** — merriam-webster.com, oed.com if your library subscribes.
- **Swedish** — svenska.se gives you SAOL, SO and SAOB in one place.
- **The Elements of Style** — the 1918 edition is out of copyright and freely available at Project Gutenberg.
- **Encyclopaedia** — britannica.com.
- **Reference manager** — Zotero (free) or your group's choice. It exports BibT_EX and it saves you retyping.
- **Where to find papers** — DBLP for CS bibliography, arXiv for preprints, your library's proxy for everything behind a paywall.

All four URLs on the 1998 version of this slide are now dead.

Books to read

- Justin Zobel. *Writing for Computer Science*. 3rd edition, Springer, 2014. The one to buy first. Written for exactly this course.
- Edward R. Tufte. *The Visual Display of Quantitative Information*. 2nd edition, Graphics Press, 2001.
- Leslie Lamport. *L^AT_EX: A Document Preparation System*. 2nd edition, Addison-Wesley, 1994.
- Frank Mittelbach and Ulrike Fischer. *The L^AT_EX Companion*. 3rd edition, Addison-Wesley, 2023.
- William Strunk Jr. and E. B. White. *The Elements of Style*. 4th edition, Longman, 1999.
- Donald E. Knuth, Tracy Larrabee and Paul M. Roberts. *Mathematical Writing*. MAA, 1989. Still free online, still excellent.